

提高作业

2.1 完整部署流程实践

按照操作脚本，完成从训练到 MuJoCo 部署的完整流程：

- 启动训练并等待至少生成一个 checkpoint;
- 导出 ONNX 模型 (`./unitree_rl_lab.sh -p --checkpoint ...`);
- 将 `policy.onnx` 和 `deploy.yaml` 部署到 `g1_ctrl` 的配置目录;
- 启动 MuJoCo 仿真器 + `g1_ctrl` 控制器;
- 通过键盘控制机器人移动。

提供以下验证材料：

- MuJoCo 仿真器运行截图 1 张：能看到 G1 机器人;
- `g1_ctrl` 终端截图 1 张：能看到 FSM 启动日志和状态切换信息;
- 机器人行走截图或录屏：展示通过 W/A/S/D 控制机器人移动。

2.2 部署架构理解

- 画出 MuJoCo Sim2Sim 部署的架构示意图（手绘、文字描述、流程图均可），标注以下要素：
 - 两个进程（`unitree_mujoco`、`g1_ctrl`）及其各自的角色
 - DDS 通信的方向和内容
 - `policy.onnx` 和 `deploy.yaml` 在哪个进程中被加载
- `config.yaml` 中 `domain_id` 的作用是什么？如果 `unitree_mujoco` 和 `g1_ctrl` 的 `domain_id` 设置不一致，会发生什么？
- `enable_elastic_hand` 的作用是什么？启动时为什么要先在 MuJoCo 窗口中按 8 把机器人放到地面，然后再启动 `g1_ctrl`？如果跳过这一步直接启动 `g1_ctrl` 会怎样？

2.3 对比分析：不同 checkpoint 的部署效果

- 选择 2 个不同的训练 checkpoint（一个迭代次数较少的，如 `model_100.pt`；一个迭代次数较多的），分别导出 ONNX 并部署到 MuJoCo;
- 对比两者的行走效果，回答以下问题：
 - 迭代次数少的模型表现如何？（能否站立？能否行走？姿态是否合理？）

- 迭代次数多的模型表现如何？
- 这说明了什么？训练迭代次数对部署效果有什么影响？
- 提供 TensorBoard 曲线截图辅助说明训练过程中的 reward 变化趋势。

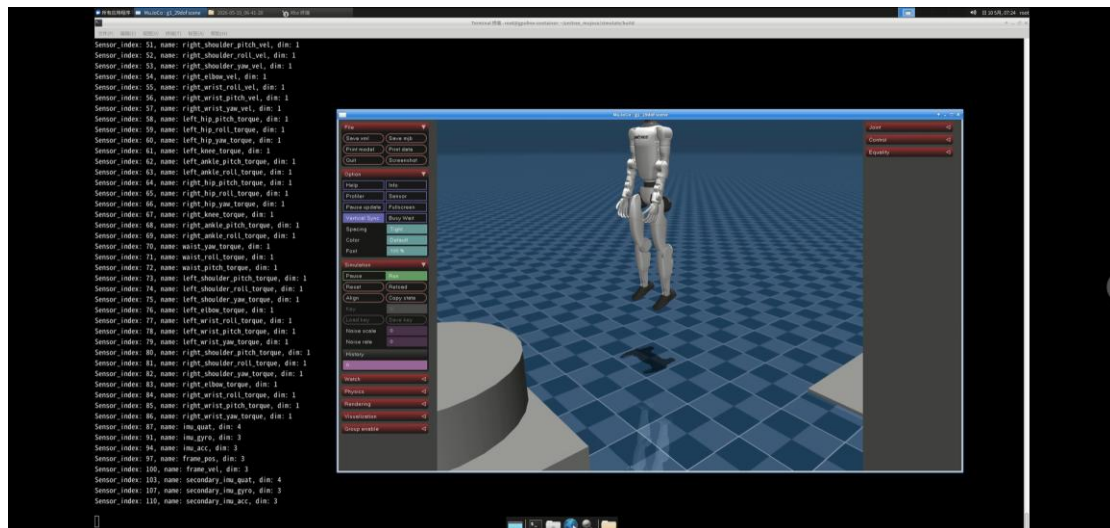
提高作业提交要求

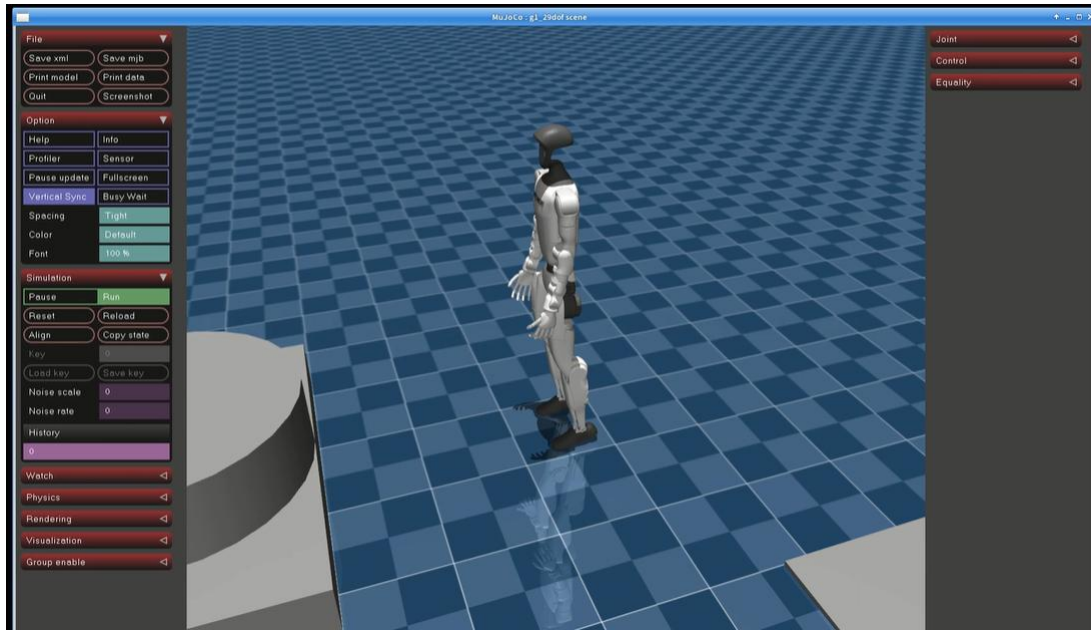
请将以上内容整理为一份 PDF，包含操作截图、架构图、对比分析以及问题的回答。

Answers:

2.1 完整部署流程实践

(1) MuJoCo 仿真器运行截图





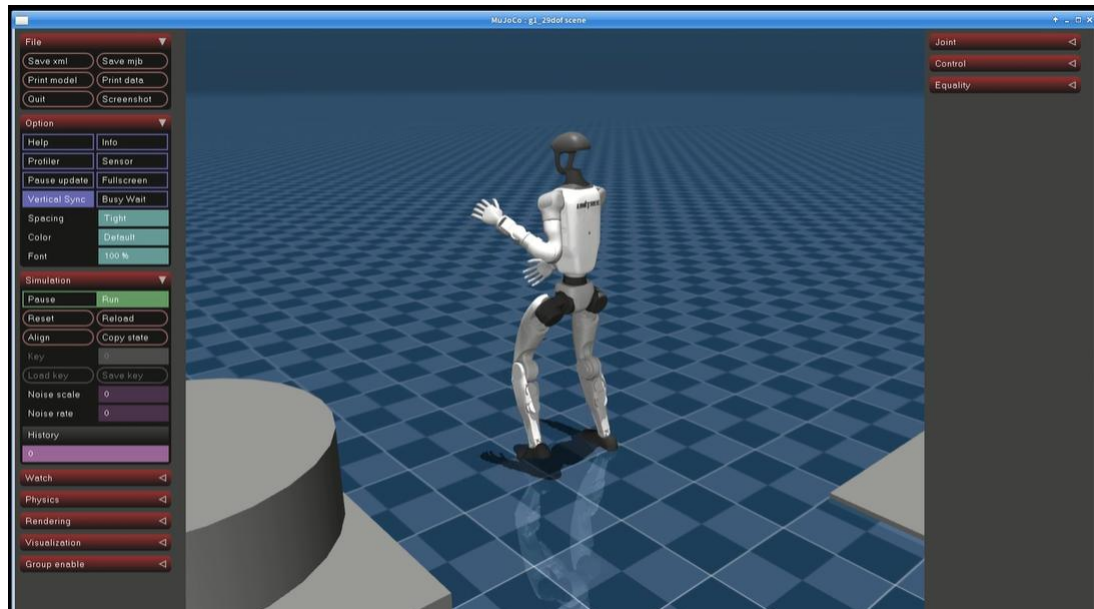
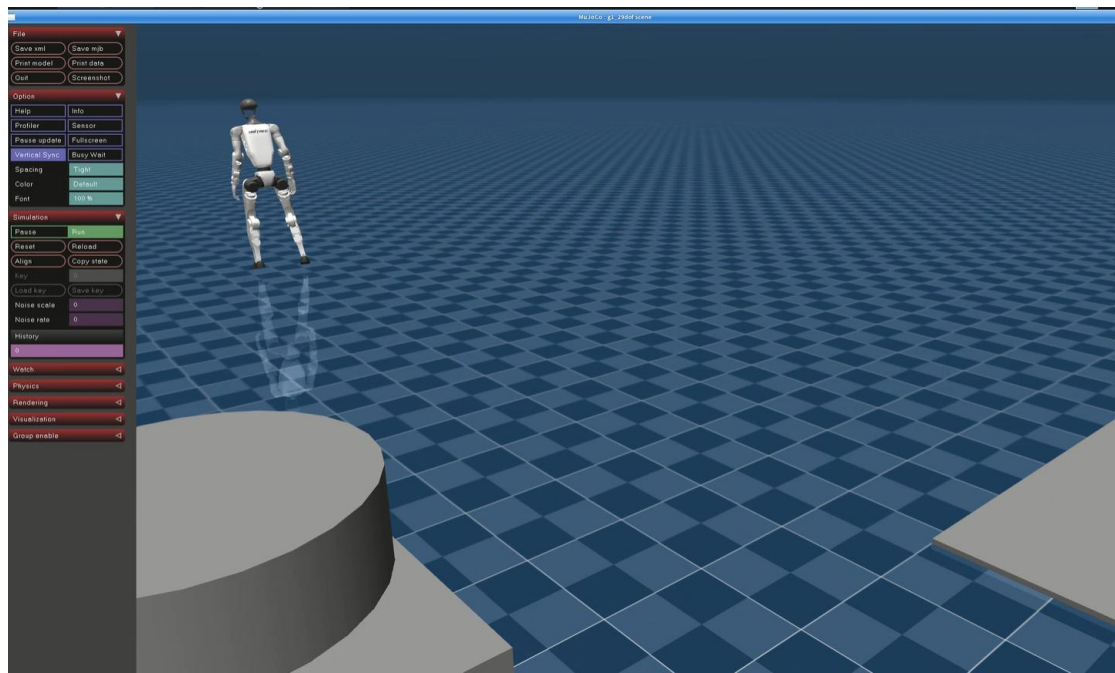
(2) g1_ctrl 终端截图

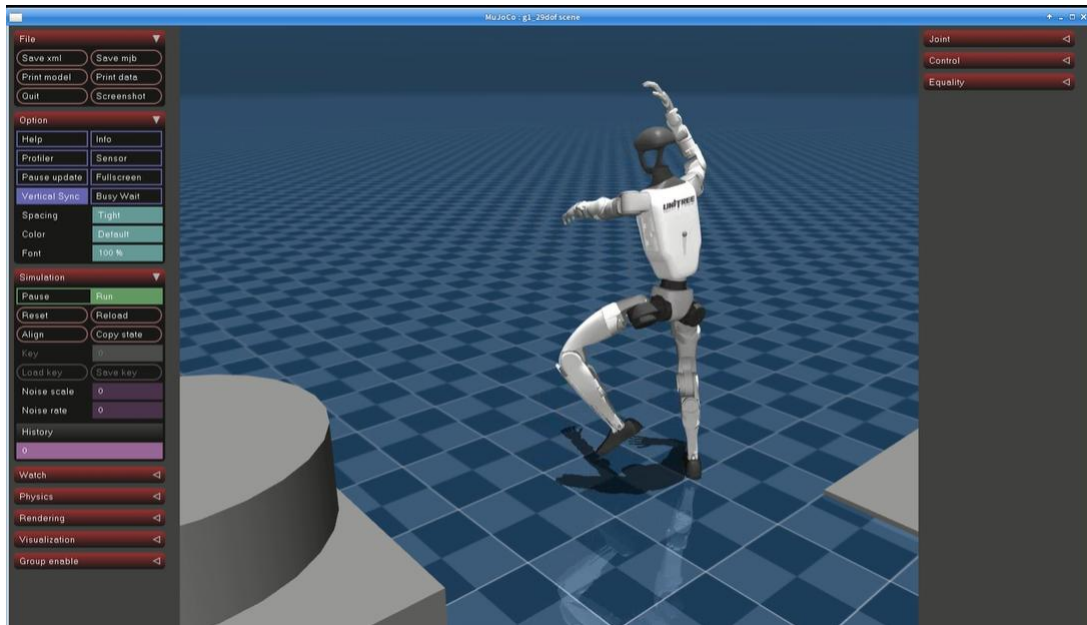
```
(isaac@lab) root@gpufree-container:~/unitree_rl_lab/deploy/robots/g1_29dof/build# ./g1_ctrl --network lo
--- Unitree Robotics ---
G1-29dof Controller
1778369610.220482 [0] g1_ctrl: selected interface "lo" is not multicast-capable: disabling multicast
[2026-05-10 07:33:30.523] [info] Waiting for connection to robot...
[2026-05-10 07:33:30.723] [info] Connected to robot.
[2026-05-10 07:33:30.823] [info] Initializing State_Passive ...
[2026-05-10 07:33:30.823] [info] Initializing State_FixStand ...
[2026-05-10 07:33:30.823] [info] Initializing State_Velocity ...
[2026-05-10 07:33:30.823] [info] Policy directory: /root/unitree_rl_lab/deploy/robots/g1_29dof/config/policy/velocity/v0
[2026-05-10 07:33:30.871] [info] Initializing State_Mimic_Dance_102 ...
[2026-05-10 07:33:30.871] [info] Policy directory: /root/unitree_rl_lab/deploy/robots/g1_29dof/config/policy/mimic/dance_102/
[2026-05-10 07:33:30.905] [info] Loaded motion file 'G1_Take_102.bvh_60hz' with duration 29.15s
[2026-05-10 07:33:30.914] [info] Initializing State_Mimic_Gangnam_Style ...
[2026-05-10 07:33:30.914] [info] Policy directory: /root/unitree_rl_lab/deploy/robots/g1_29dof/config/policy/mimic/gangnam_style/
[2026-05-10 07:33:31.010] [info] Loaded motion file 'G1_gangnam_style_V01.bvh_60hz' with duration 32.37s

===== 已加载的状态列表 =====
状态 ID: 1 | 状态名称: Passive
状态 ID: 2 | 状态名称: FixStand
状态 ID: 3 | 状态名称: Velocity
状态 ID: 101 | 状态名称: Mimic_Dance_102
状态 ID: 102 | 状态名称: Mimic_Gangnam_Style
=====

[2026-05-10 07:33:31.019] [info] FSM: Start Velocity
Press [L2 + Up] to enter FixStand mode.
And then press [R1 + X] to start controlling the robot.
[2026-05-10 07:34:15.367] [info] FSM: Change state from Velocity to FixStand
[2026-05-10 07:34:22.130] [info] FSM: Change state from FixStand to Velocity
[2026-05-10 07:34:46.590] [info] FSM: Change state from Velocity to FixStand
[2026-05-10 07:34:57.137] [info] FSM: Change state from FixStand to Passive
[2026-05-10 07:34:59.921] [info] FSM: Change state from Passive to FixStand
[2026-05-10 07:35:01.277] [info] FSM: Change state from FixStand to Velocity
[2026-05-10 07:35:11.618] [info] FSM: Change state from Velocity to Passive
[2026-05-10 07:35:17.396] [info] FSM: Change state from Passive to FixStand
[2026-05-10 07:35:30.329] [info] FSM: Change state from FixStand to Mimic_Dance_102
[2026-05-10 07:35:30.580] [info] FSM: Change state from Mimic_Dance_102 to Mimic_Gangnam_Style
[2026-05-10 07:35:36.612] [info] FSM: Change state from Mimic_Gangnam_Style to Passive
[2026-05-10 07:35:39.181] [info] FSM: Change state from Passive to FixStand
[2026-05-10 07:35:41.827] [info] FSM: Change state from FixStand to Velocity
[2026-05-10 07:35:47.307] [info] FSM: Change state from Velocity to Mimic_Dance_102
[2026-05-10 07:36:05.153] [info] FSM: Change state from Mimic_Dance_102 to Mimic_Gangnam_Style
[2026-05-10 07:36:07.398] [info] FSM: Change state from Mimic_Gangnam_Style to FixStand
[2026-05-10 07:36:08.582] [info] FSM: Change state from FixStand to Passive
```

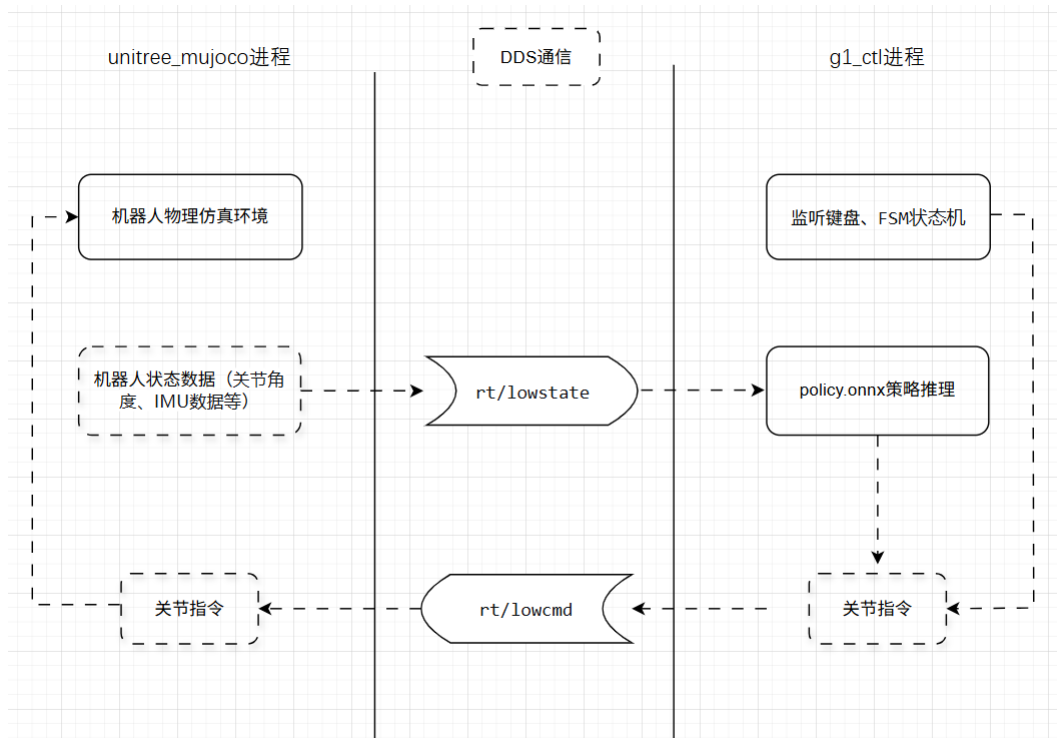
(3) 机器人行走截图





2.2 部署架构理解

(1) MuJoCo Sim2Sim 部署架构示意图



说明：

1. unitree_mujoco 进程负责物理仿真，将机器人状态（关节角度、IMU 数据等）通过 rt/lowstate topic 发布出去；
2. g1_ctl 进程订阅 rt/lowstate，获取最新的机器人状态作为观测输入，输入 policy.onnx 进行推理，得到目标动作（关节指令），再通过 rt/lowcmd topic 发布

给 unitree_mujoco;

3. unitree_mujoco 订阅 rt/lowcmd, 将收到的关节指令作为仿真中关节的 PID 控制目标, 驱动仿真机器人运动。
4. policy.onnx 与 deploy.yaml 全部在 g1_ctl 进程中加载。

(2) config.yaml 中 domain_id 的作用是什么? 如果 unitree_mujoco 和 g1_ctl 的 domain_id 设置不一致, 会发生什么?

domain_id 是 DDS 通信中的域标识符, 用于在同一网络上隔离不同 DDS 通信组。如果两边 domain_id 不一致, 则仿真器和控制程序无法进行通信。

(3) enable_elastic_band 的作用是什么? 启动时为什么要先在 MuJoCo 窗口中按 8 把机器人放到地面, 然后再启动 g1_ctl? 如果跳过这一步直接启动 g1_ctl 会怎样?

enable_elastic_band 控制 MuJoCo 场景里是否启用“弹性带”, 为仿真机器人提供临时支撑。仿真启动后, 先按 8 把机器人放到地面, 然后再启动 g1_ctl, 这样能让机器人从“双脚着地、身体被弹性带轻托”的稳态下尽量平滑进入到 Velocity。如果跳过这一步直接启动 g1_ctl, 机器人很可能会重重摔倒。

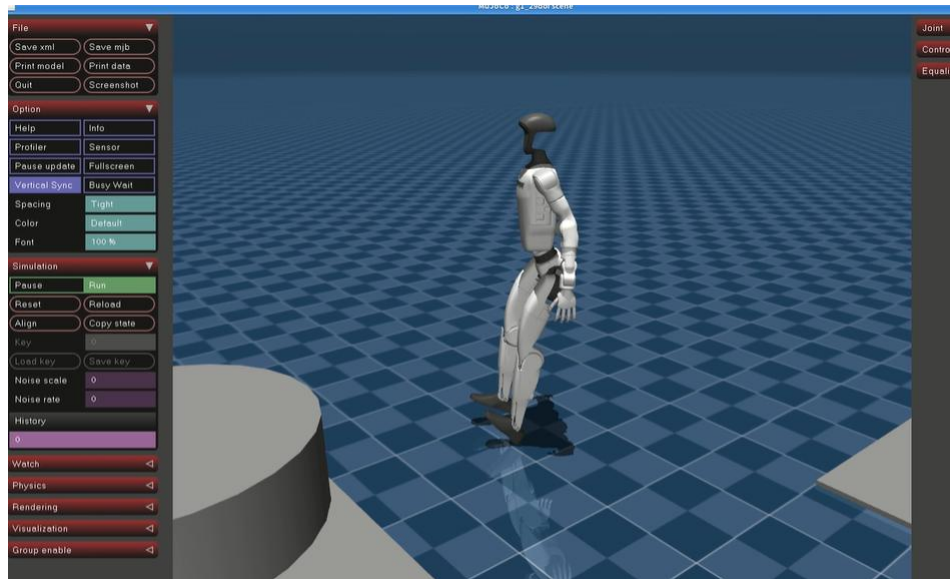
2.3 对比分析: 不同 checkpoint 的部署效果

- 选择 2 个不同的训练 checkpoint, 分别导出 ONNX 并部署到 MuJoCo;

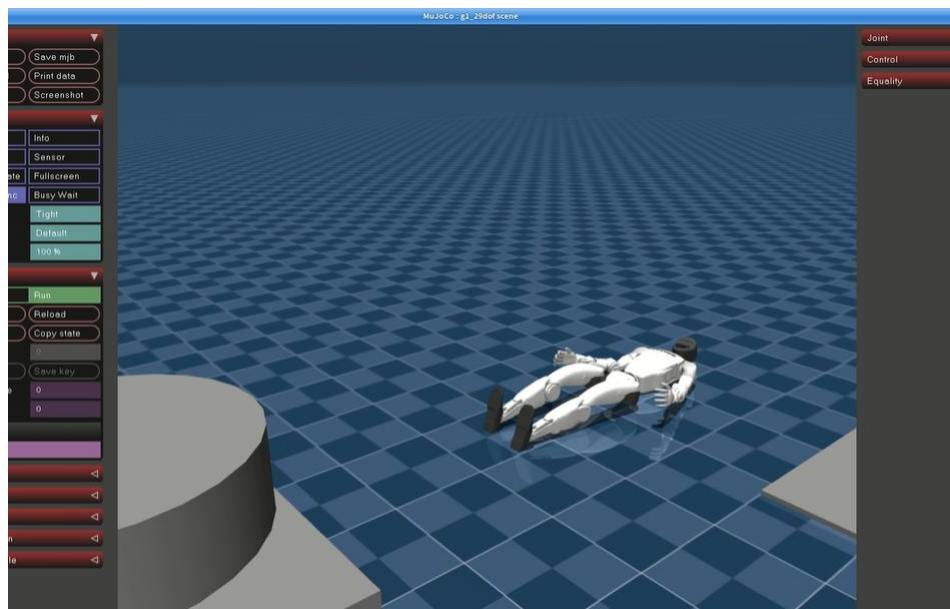
model_100.pt、model_800.pt

- 对比两者的行走效果, 回答以下问题:
 - 迭代次数少的模型表现如何?

model_100.pt 对应的机器人在运行 g1_ctl 后, 姿态就有些不稳。



在松开绑带后，即发生摔倒。



- 迭代次数多的模型表现如何？

model_800.pt 对应的机器人，相比较迭代次数 100 的，能够正常站立，姿态正常，截图参见 2.1 部分。

- 这说明了什么？训练迭代次数对部署效果有什么影响？

说明随着训练迭代次数的增加，机器人整体稳定性和性能都逐渐提高。从下图的 TensorBoard 曲线也能看出类似趋势，比如每轮 episode 的平均奖励随着迭代次数的增加而增加。

- 提供 TensorBoard 曲线截图辅助说明训练过程中的 reward 变化趋势。

